Data structures & Algorithms
-00940224-

- **Student:** Suheil Khourieh – [redacted]

Question 1:

Solution:

We will convert and look at the problem as a graph, where the vertices are the bus stops and the edges between the vertices are the one way paths between the bus stops, it is given that $|bus\ stops| = n\ and\ |one\ way\ paths| = m$, the idea is to construct the graph $G = (bus\ stops, one\ way\ paths)$, and add to the graph a new vertex s that will be connected to all the bus stops of the faculties the captain has to visit (the set A), then by running $BFS(G, s)$ we can easily find the faculty that the captain has to end his trip at (from the set B) and by traveling through the path using the π attribute, we can find the closest vertex from the set A to the vertex. Here is the pseudo-code to further explain the solution:

```
ShortestPathFromA2B(A, B, bus stops, one way paths)
1: source = new vertex
2: newEdges = new set of edges connecting source with all the vertices in A
3: G = new graph(bus stops ∩ {s}, one way paths ∩ newEdges)
4: BFS(G, s)
5: endingStop = {}
6: minimal_length = ∞
7: for all b ∈ B do
8:     if b.d < minimal_length then
9:         endingStop = b
10: startingStop = {}
11: a = b.π
12: while a ≠ NIL do
13:     if a ∈ A then
14:         startingStop = a
15:         break
16:     a = a.π
17: return (startingStop, endingStop, b.d − 1)
```

We assumed that every vertex has d and π attributes, exactly how we saw in the lectures.
The loop line 7-9 is done to find the vertex from the set B that has the minimal d value, this is done in $O(|B|) = O(|V|)$.
The loop line 12-16 is done to find the vertex from the set A that is closest to the vertex found earlier (from the set B), this is also done in $O(|A|) = O(|V|)$.

Correctness:

We need to prove that the path we got mark it with P, does indeed describe the shortest path between any stop in A and any stop in B, suppose there exists a different path mark it with P' that describes a shorter path between any vertex from A to any vertex in B.
$P' =< a', \dots, b' >$ and $P =< a, \dots, b >$, assume by contradiction that $|P'| < |P|$, according to the BFS algorithm and based on how we choose b, $b.d \leq b'.d$, and we know that $|P| = b.d - 1$, $|P'| = b'.d - 1$, thus $|P| \leq |P'|$, for every $a' \in A, b' \in B$. ■
In a similar way we will prove that $|< a, \dots, b >| \leq |< a', \dots, b >|, \forall a' \in A$, assume by

contradiction that there exists $a' \in A$ where the path $|< a', \dots, b >|$ is smaller than $|P|$ we already found, but because the path was based on a BFS search, b was choosen as the closest to the added vertex $source$ and $source$ is one step away from any vertex in A, thus the path found indeed describes the shortest path between any pair of vertices from A and B.
The length of the path the captain has to take is $b.d - 1$, simply because we added the vertex $source$.

Time-complexity:
The total run-time of the algorithm is $\mathrm{O}(V + E) = \mathrm{O}(n + m)$ exactly how we were asked, this is true because all we did is use BFS, some $\mathrm{O}(1)$ operations and two loops that take $\mathrm{O}(G.V)$ each, hence in total we have $\mathrm{O}(n + m)$.

Question 2:

Definition: Given a directed graph $G = (V, E)$, a tour is a path (not necessarily simple) in G that passes all the vertices in V.

Part 1:

Proposed an algorithm that takes as an input a $DAG\ G = (V, E)$ and returns a tour in G if such exists, and a message that there doesn't exist one otherwise. The proposed algorithm should take $O(V + E)$ in time complexity.

Solution:

Before proposing an algorithm that does the required, we will propose a helping function $findEdge(G, v, u)$ that takes two vertices from G and returns true if there exists an edge from v to u, false otherwise.

```
findEdge(G, v, u)
for all w ∈ G.Adj[v] do
    if w == u then
        return True
return False
```

We can easily see that this function works as intended, and that it takes $O(d_v)$ where d_v is the number of outgoing edges v has.

Not let's proceed with proposing an algorithm that does the asked in the question. The idea behind the algorithm is to create a topological sort of the graph G and then iterate over the topological sort in order to make sure that every pair of following vertices in topological sort have an edge in the appropriate direction connecting them. Here's the pseudo-code of the topological sort:

```
Topological_sort(G)
for all u ∈ G.V do
    u.color = white
path = new linked list
for all u ∈ G.V do
    if u.color == white then
        Visit(G, u, path)
return path
```

```
Visit(G, u, path)
u.color = gray
for all v ∈ G.Adj[u] do
    if v.color == white then
        Visit(G)
u.color = black
path.appendAtStart(u)
```

You can see that $Topological_sort(G)$ looks very similar to the function $DFS(G)$ taught in the lectures and that $Visit(G, u, path)$ also looks similar to $DFS_Visist(G, u)$ taught in the lectures too, those are the modified versions of them in order to answer our needs. The modifications done are so minimal, a few $O(1)$ operations were removed from the original versions and a few $O(1)$ were added, in total the run-time complexity did not change and it is still $O(|V| + |E|)$.

The path returned right before the elimination of $Topological_sort(G)$ is the topological sort of the $DAG\ G = (V, E)$.

We can easily prove that this is indeed the topological sort of the graph, but in tutorial 5 we already saw that this is how a topological sort of a DAG is constructed.

Now all we need to do is iterate over the list received and make sure that every pair of

following vertices have an appropriate edge connecting them, to do so, we will introduce the final function that answers everything that was asked in the question.

```
find_tour(G)
tour = Topological_sort(G)
for all v ∈ tour/{last vertex} do
    if findEdge(G, v.getValue(), v.next().getValue()) == False do
        return "There does not exist a tour in G"
return tour
```

Now we should prove the correctness and the time-complexity of the algorithm:

Correctness:

We know that for every DAG, there exists a topological sort, we have already shown above that the suggested algorithm $Topological_sort(G)$ indeed returns a topological sort of the graph, we know that the topological sort includes all the vertices in the graph. Thus, no matter what the graph is, as long as it is acyclic and directed, we will always get a list containing every vertex in the graph exactly once.

Let the topological sort calculated be marked with $S = v_1, \dots, v_n$.

We now need to check every pair of following vertices v_i, v_{i+1} in $S, \forall i = 1, \dots, n-1$ and make sure that the edge $(v_i, v_{i+1}) \in E$, to do so we will iterate over S and every time call the function $findEdge$ that was proposed at the start. If an edge doesn't exists, "$There\ does\ not\ exist\ a\ tour\ in\ G$", otherwise the topological sort is returned and it is the tour described in the question.

We should prove now that $there\ exists\ a\ tour\ in\ G \Leftrightarrow (v_i, v_{i+1}) \in E, \forall i = 1, \dots, n-1$, in other words, we should prove that there exists a tour in G if and only if the topological sort is also a legal tour.

$\Leftarrow$:

If every edge $(v_i, v_{i+1}) \in E$, then the topological sort is the tour we are looking for, this is true because of how the topological sort is created, it always includes all the vertices no matter what, thus the sort calculated will include all the vertices, and because all the edges are legal, then the sort calculated is indeed a legal tour.

$\Rightarrow$:

$There\ exists\ a\ tour\ in\ G \Rightarrow (v_i, v_{i+1}) \in E, \forall i = 1, \dots, n-1$

Assume by contradiction that there exists a tour in G, but not all the edges do exist, in other words, assume by contradiction that there exists a tour in G but $\exists (v_i, v_{i+1}) \notin E$

Let T be a tour in G and $(v_i, v_{i+1}) \notin E$.

By the definition of topological sort, v_{i+1} must appear after v_i in the topological sort, the edge connecting them does not exist, thus v_i is not connected to v_{i+1}.

T is a legal path, and the pair of vertices v_i and v_{i+1} are connected there, if v_{i+1} appears before v_i in T then it also must appear before it in the topological sort ($\rightarrow\leftarrow$).

If v_{i+1} appears after v_i in T, then there exists a path connecting them, thus in the topological sort the edge must be legal ($\rightarrow\leftarrow$).

Time-complexity:
We can easily see that the function $find_tour$ takes a total of $O(|V| + |E|)$, let's prove this:
line 1 takes $O(|V| + |E|)$, this is true because the call to $Topological_sort$ is so similar to the function DFS, the differences are some $O(1)$ operations removed and some added, we have shown in the lectures that DFS takes $O(|V| + |E|)$, thus $Topological_sort$ is also $O(|V| + |E|)$.
The loop lines 2-4 iterates over all the vertices but the last one in the tour, and makes sure that every pair of following vertices have an edge connecting them in G, every vertex $v \in V$ has d_v edges in total, assume for each vertex we iterate over all it's edges, thus in total the loop iterates $\sum_{v \in V} d_v$, which is exactly $|E|$.
In total we have $O(|V| + 2|E|) \Rightarrow O(|V| + |E|)$.
This check is not necessary, because as mentioned above, this check can be done while creating the path, we choose to use this check because it doesn't affect the time-complexity and it makes the solution clear.

Part 2:
Propose an algorithm that takes a directed graph $G = (V, E)$ as an input and returns if there exists a tour in G or not (no need to return the tour itself). The algorithm should run in $O(V + E)$ time-complexity.

Solution:
The suggested algorithm will use the $component\ digraph$ taught in the lectures. We will create a component digraph from the graph G, let the result be marked with G^s. According to a lemma proven in the lecture, G^s is DAG, thus the algorithm proposed in the first part can be used on G^s, check if there exists a tour in G^s by running the algorithm from the first part, if there exists such tour, return that there exists a tour in G, otherwise return that there doesn't.

Correctness:
$\Rightarrow$: $\exists tour\ in\ G \Rightarrow \exists tour\ in\ G^s$
Assume by contradiction that $\exists tour\ in\ G\ and\ \nexists tour\ in\ G^s$.
Let T be the tour in G, can easily see that every SCC in G will appear together as a group in T, in other words, every vertex of an SCC in G will appear next to the rest of the vertices of the same SCC, this is true simply because every set of SCC is "one way" connected to another SCC if such set exist, in other words, if any pair of $SCCs$ have a path connected them the path is "one way" and there cant be any cycles between them.
By definition, every SCC is a **strongly connected component set** meaning there must be tour for every SCC.
Thus, T will be a tour between all the $SCCs$ while every SCC appears together in T.
But according to the assumption, G^s doesn't have a tour ($\rightarrow\leftarrow$).

$\Rightarrow G^s$ must have a tour.

$\Leftarrow$: $\exists tour\ in\ G^s \Rightarrow \exists tour\ in\ G$
Let T^s be the tour in G^s, thus T^s is a path connecting all the $SCCs$ of G, and we know that every SCC in G is a strongly connected component set, in other words, for every SCC there exists a tour for it, let's construct a path T in the following way:
For every SCC in G find a tour for it, add it to T, connect the tour created with the next tour from the next SCC using the relevant edge from T^s. $\Rightarrow T$ is a tour for G.

We have shown that $\exists tour\ in\ G \Leftrightarrow \exists tour\ in\ G^s$.■

Time-complexity:
G^s can be constructed in $O(V + E)$ according to the lecture, the result is a directed graph $G^s = (V^s, E^s)$ where $|V^s| \leq |V|$ and $|E^s| \leq |E|$, thus running the algorithm from the fisrt part is indeed $O(V + E)$.
In total we have $O(V + E)$. ■

Question 3:

Given an undirected and connected graph $G = (V, E)$, a breaking edge in an edge that after removal, the graph G will no longer be connected. G is called breakable if there exists in G a breaking edge. This question engages in the decision of if the graph G is breakable or not. After running $DFS(G)$, let the directed graph $\vec{G} = (\vec{V}, \vec{E})$ where $\vec{V} = V$ and the direction of the edges are according to the direction they were first discovered while running $DFS(G)$. We should notice that the graph $\vec{G}$ is the graph G_π with the directed edges from the parent to the child with the addition of back edges. Thus, it is possible to use the definition of descendant and ancestor also in $\vec{G}$. For example, the vertex u is an ancestor of the vertex v in $\vec{G}$ if and only if u is an ancestor of v in G_π.

Part 1:

Suggest an algorithm that takes an undirected and connected graph $G = (V, E)$ as an input and returns $\vec{G}$ as an output. The time-complexity of the suggested algorithm should be $O(V + E)$.

Solution:

We will suggest an algorithm that is based on the DFS algorithm introduced in the lectures, it works in a similar way, there is some minimal changed done in the function $DFS(G)$ and some done in $DFS_Visit(G, u)$.

We will initialize an empty set $newE$ that will hold all the relevant edges for the graph $\vec{G}$.

The second change is that in the if statement lines 5-6 we will add an $O(1)$ operation that adds the edge (u, v) to the set $newE$.

Another change in the function $DFS_Visit(G, u)$ is that we will add another if statement in the for loop that will check if $v.color \neq white\ and\ u.\pi \neq v$, if the condition is true, then the edge (u, v) will be added to the set of directed edges $newE$.

To further explain the changes done, the following pseudocode describes the changes suggested above while changing the name of the function DFS to be $Directed_DFS$ and DFS_Visit to be $Directed_DFS_Visit$:

```
Directed_DFS(G)
1: for all u ∈ G.V do
2:     u.color = white
3:     u.π = NIL
4: time = 0
5: newE = ∅
6: for all u ∈ G.V do
7:     if u.color == white do
8:         Directed_DFS_Visist(G, newE, u)
9: G⃗ = (V, newE)
10: return G⃗
```

```
Directed_DFS_Visit(G, newE, u)
1: time = time + 1
2: u.d = time
3: u.color = gray
4: for all v ∈ G.Adj[u] do
5:     if v.color == white then
6:         v.π = u
7:         newE = newE ∪ (u, v)
8:         Directed_DFS_Visit(G, newE, u)
9:     if v.color ≠ white and u.π ≠ v then
10:        newE = newE ∪ (u, v)
11: u.color = black
12: time = time + 1
13: u.f = time
```

Part 2:

Prove that every breaking edge in G will be a tree edge in $DFS(G)$.

Solution:

We should prove that every breaking edge in G will be a tree edge after running $DFS(G)$.

First of all, according to the features of DFS, the result of running the algorithm is a tree if and only if the graph G is connected. This fact will be used later to prove the lemma.

Let's assume by contradiction that there exists a connected undirected graph $G = (V, E)$ where there exists a breaking edge $e_0 = (u, v)$ and e_0 is not a tree edge in the graph we get after running $DFS(G)$.

Let G_π be the graph we got as a result of running DFS on G.

If e_0 is not a tree edge, then in G_π ,v will be connected to the vertex we started the search from if and only if there exists a tree edge $e_1 = (v, a)$ where $a \neq u$.

If the edge $e_1$doesn't exist, it means that the vertex v is not connected to the vertex we started the search from implying that the graph G_π is not connected thus it can't be a tree $\rightarrow$ G in the first place is not connected which contradicts how G was defined in the question.

If both the edges e_0 and e_1 exist in the graph G, and the vertex v is connected to the vertex we started the search from and G_π is a tree and not a forest, then there exists a path from the vertex we started the search from to the vertex v, let that path be p_0.

p_0 exists in $G_\pi \rightarrow p_0$ exists in G.

Thus, removing the edge e_0 will not disconnect v from the graph and the graph will still be connected, which contradicts the assumption we made.

We have reached a contradiction $\rightarrow$ our assumption can't be true $\rightarrow$ the claim it true ■.

Part 3:

Prove: edge (u, v) that was defined as a tree edge after running $DFS(G)$ is a breaking edge in G if and only if there is no back edge from a descendant of v to an ancestor of u in $\vec{G}$.

Solution:

We should prove that the edge (u, v) that was identified as a tree edge after running $DFS(G)$ is a breaking edge in G if and only if there doesn't exist a back edge from a descendant of v to an ancestor of u in $\vec{G}$.

First of all, let's prove that the tree edge (u, v) that was identified after running $DFS(G)$ is a breaking edge in $G \Rightarrow$ there doesn't exist a back edge from a descendant of v to an ancestor of u.

Let's assume by contradiction that the claim is not true, in other words, let's assume by contradiction that the edge $e_0 = (u, v)$ was identified as a breaking edge after running $DFS(G)$ and that e_0 is a breaking edge in G and that there **exists** a back edge from a descendant of v to an ancestor of u. Let the descendant of v be marked as s, and the ancestor of u be marked with m. <u>It is possible that $v = s$ or $u = m$.</u>

Let the back edge than connects s and m be called e_1.

Let the path connecting m with u in G be called p_1.

Let the path connecting s with v in G be called p_2.

Let G' be the graph G after removing the edge e_0.

We know that G' should not be connected since it is the result from removing the breaking edge e_0 from the graph G, in other words, G' is not connected because there isn't a path connecting the vertices u and v. But by chaining p_1 with e_1 and $p_2 (p_1 \rightarrow e_0 \rightarrow p_2)$ we will get a path connecting u with v thus G' is still connected and e_0 is not a breaking edge $(\rightarrow\leftarrow)\blacksquare$.

$\Rightarrow$ The claim is true.

Let's prove the other direction: There doesn't exist a back edge from a descendant of v to an ancestor of u in $\vec{G} \Rightarrow$ the tree edge (u, v) is a breaking edge in G.

Let's assume by contradiction that the claim is not true, in other words, let's assume that there doesn't exists a back edge from a descendant of v to an ancestor of u in $\vec{G}$ and that the tree edge (u, v) is not a breaking edge in G.

Let the edge (u, v) in G be marked with e_0 and the directed edge (u, v) in $\vec{G}$ be marked with $\overrightarrow{e_0}$.

e_0 is not a breaking edge, thus there exists a path from u to v that doesn't include e_0, let that path be called p_1 in G.

Let $\overrightarrow{p_1}$ be the path p_1 but in $\vec{G}$.

We know that $\overrightarrow{p_1}$ doesn't include $\overrightarrow{e_0}$ because of how we choose p_1.

If p_1 exists after running $DFS(G)$ as a path that includes only tree edges, it implies that e_0 is not a tree edge after running $DFS(G)$ this is true because of how DFS works, it searches depth first and since the path p_1 include only tree edges, it means that e_0 will be a back edge, which contradicts what was given in the question.

If p_1 includes back edges after running $DFS(G)$, this implies that e_0 will exist as a tree edge after running $DFS(G)$ and p_1 will be connecting v and u using some back edges,

thus $\overrightarrow{p_1}$ also connects v with u in $\vec{G}$.
$\overrightarrow{p_1}$ includes some back edges from a descendant of v to an ancestor of u which contradicts our assumption. (→←) ■. ⇒ The claim is true.

Part 4:
Suggest an algorithm that takes an undirected connected graph $G = (V, E)$ and decides if G is breakable.

Solution:
We know that a directed graph is breakable if and only if the graph is not strongly connected.
A graph is strongly connected if and only it has exactly one strongly connected component.
Thus, a directed graph is breakable if and only if it has exactly one strongly connected component.
The algorithm we will suggest checks if the graph is breakable by checking if the directed version of the graph is breakable.
Let $\vec{G}$ be the directed version of the undirected graph G. We get $\vec{G}$ by running the algorithm suggested in the first part, this operation is done once and takes $O(V + E)$.
Now we can check if $\vec{G}$ is strongly connected by running the algorithm SCC introduced in lectures that works in $O(V + E)$.
SCC is an algorithm that returns all the strongly connected components sets for the provided directed graph.
Thus the result of running SCC on $\vec{G}$ will be all the strongly connected components sets there is for $\vec{G}$.
Now we check how many strongly connected components sets there is, if there is exactly one or zero (incase G was an empty graph) we return that G is not breakable.
If there is more than one strongly connected component set, we return that G is breakable.
The operation of finding the number of strongly connected components sets can be done in $O(1)$ if the result of running SCC is an array for example. If not (for example if the result of running SCC is a linked list) we can find the number of the strongly connected component sets by irritating over the return data structure and counting, this can be done at $O(V)$.
To sum everything up, we first find $\vec{G}$ by running the algorithm suggested above, then we run SCC on $\vec{G}$, we return that G is breakable if and only if the size of the retuned result of running SCC on $\vec{G}$ is greater than 1.
Time-Complexity:
Finding $\vec{G}$ is done once and takes $O(V + E)$
Running SCC on $\vec{G}$ is done once and takes $O(V + E)$
Finding the size of the result of running SCC is done once. Depends on what data structure is used to represent the sets, either way it will take $O(V)$ at most and $O(1)$ at the least, let's assume it takes $O(V)$.
In total → $O(V + E) + O(V + E) + O(V) = O(3V + 2E) = O(V + E)$ ■.

Question 4:
Let $G = (V, E)$ be an acyclic directed graph (DAG).

Part 1:
Prove that the linked list created from running the following solution provides a topological sort for G.

A. Initialize an empty list L.
B. Find all the vertices with an in degree that is equal to 0.
   i. At them to the end of the list L.
   ii. Delete them and all the edges that comes out of them from the graph G.
C. While the graph G is not empty, return to step B.
D. Return at L.

Solution:
We should prove that after we are done running all the sections (A – D), L will describe a topological sort of $G = (V, E)$, where G is DAG.
Let's assume by contradiction that there exist two vertices $v, u \in V$ where $u \neq v$ and $(u, v) \in E$ and v appears before u in the list L.
We know that for any vertex, it's in-degree can only get smaller while running the algorithm, in other words, the in-degree of any vertex can't get greater while running the algorithm.
According to our assumption:

- $(u, v) \in E \rightarrow$ at some point in the algorithm, $u.inDegree$ will be zero for the first time, let that time be t_0. At time t_0 $v.inDegree$ is greater than zero.
  In other words, at time t_0 the following is true: $u.inDegree = 0 \wedge v.inDegree > 0$.
  This is true because at time $t_0, u.inDegree$ will be zero for the first time, indicating that the edge (u, v) still exists, thus $v.inDegree$ is at least $1 \rightarrow v.inDegree > 0$.
- v appears before u in $L \rightarrow$ at the time of entering v to the list, it's inDegree was zero and u's inDegree was greater or equal to zero. In other words:
  Let t_1 be the timestamp where v is entering the list, we know that $v.inDegree = 0$ and that $u.inDegree \geq 0$.

If at time t_1, $u.inDegree = 0$, it means that there is no edge connecting u and v which contradicts our assumption.
If at time t_1, $u.inDegree > 0$, it means that there can't be an edge (u, v) according to the first bullet-point, because if there was (u, v) $u.inDegree$ will be zero before $v.inDegree$ according to the first bullet-point.
$(\rightarrow\leftarrow)$
Thus, for any two vertices $u \neq v \in V$, $(u, v) \in E \rightarrow u\ will\ appear\ before\ u\ in\ L$. ■

Part 2:

Suggest an algorithm that works in $O(V + E)$ time, and actualize the solution provided in the previous section.

The algorithm that we will suggest is the same as the DFS algorithm introduced in lectures with a twist, we will rename the $DFS(G)$ algorithm to be $Topological_Sort(G)$, and $DFS_Visit(G, u)$ to be $Topological_Insertion(G, u)$.
The only difference between $DFS(G)$ and $Topological_Sort(G)$ is that in $Topological_Sort(G)$ we will initialize an empty linked list.
The only difference between $DFS_Visit(G, u)$ and $Topological_Insertion(G, u)$ is that in $Topological_Insetion(G, u)$ we will insert vertex to the linked list initialized in $Topological_Sort(G)$.
The following pseudo code shows the differences described above:

```
Topological_Sort(G)
for all u ∈ G.V do
    u.color = white
    u.π = NIL
List = ∅
time = 0
for all u ∈ G.V do
    if u.color == white then
        Topological_Insertion(G, u)

InsertAtStart(List, u)
if List.head == NIL then
    List.head = u
    return List
u.next = List.head
List.head = u
return List
```

```
Topological_Insertion(G, u)
time = time + 1
u.d = time
u.color = gray
for all v ∈ G.V do
    if v.color == white then
        v.π = u
        Topological_Insertion(G, u)
u.color = black
time = time + 1
u.f = time
List = InsertAtStart(List, u)
```

Let's prove that the following algorithm works:
The function $InsertAtStart(List, u)$ does exactly as it says, it adds the provided node u to the provided list $List$. We can also say this function chains u to $List$ in a way where u is at the start of the $List$.
We can easily see that this function works in $O(1)$.

The function $Topological_Insertion(G, u)$ works exactly as $DFS_Visit(G, u)$ that was introduced in the lectures, the only difference is the last row which is an $O(1)$ operation, thus in total it runs in $O(E)$.

The function $Topological_Sort(G)$ works exactly as $DFS(G)$ that was introduced in lectures, the only difference is a new row added between row 3 and row 4 in $DFS(G)$.
The added operation is an initialization of a linked list that takes $O(1)$.

$$\rightarrow T(Topological_Sort) = O(V + E)$$

Running the algorithm on a $G = (V, E)$ that is DAG will result in a linked list that will describe G in a way where it will be sorted in a topological way.
It will run at $O(V + E)$ where V is the set of vertices and E is the set of edges in G.

Question 5:

Let $G = (V, E)$ be a directed graph and $s \in V$ a vertex. Given that for every vertex there exists an attribute $value$. Suggest an algorithm that works in $O(V + E)$ time-complexity that calculates for every vertex $v \in V$ the total number of shortest paths from s to v in G, and the minimal sum of values in a path between all the shortest paths. i.e. the suggested algorithm should return the following values $NumMinPaths, SumMinPaths$ where:

$$MinPaths(s, v) = \arg \min_{P=<s,\dots,v>} |P|$$

$$NumMinPaths(s, v) = |MinPaths(s, v)|$$

$$SumMinPaths(s, v) = \min_{P \in MinPaths(s,v)} \sum_{w \in P} w.value$$

Solution:

We will suggest an algorithm that is based on the $BFS(G, s)$ that was introduced in the lectures, before suggesting anything we need to clarify the following:
Each vertex has the known attributes $color, d, \pi$. In addition to those attributes, we will add two more attributes, the first is $SPCounter$, this attribute will be responsible to having a counter on the total number of shortest paths every vertex v has from the vertex s. The second added attribute will be $MinimalValPath$, this attribute will be responsible to saving the minimal sum of the values in a shortest path.
The idea behind the changes done on BFS and $BFS_Initialization$ is to initialize the added attributes and keep track of them, here is the pseudo-code for the changes:
The modifications done are colored with red.

```
BFS_Modified(G, s)
Init(G, s, Q)
while Q ≠ ∅ do
    u = Dequeue(Q)
    for all v ∈ G.Adj[u] do
        if v.color == white then
            v.color = gray
            v.d = u.d + 1
            v.π = u
            v.SPCounter += 1
            v.MinimalValPath = u.MinimalValPath + v.value
            Enqueue(Q, v)
        elif v.color ≠ white and v.d == u.d + 1 then
            v.SPCounter += 1
            if v.MinimalValPath ≥ u.MinimalValPath + v.value then
                vMinimalValPath = u.MinimalValPath + v.value
    u.color = black
```

```
Init(G, s, Q)
1: for all v ∈ G.V − {s} do
2:     v.color = white
3:     v.d = ∞
4:     v.π = NIL
5:     v.SPCounter = 0
6:     v.MinimalValPath = ∞
7: s.color = gray
8: s.d = 0
9: s.SPCounter = 0
10: s.MinimalValPath = s.value
11: Q = ∅
12: Enqueue(Q, s)
```

After running $BFS_Modified(G, s)$, every vertex $v \in V$ will have attributes $v.SPCounter$ and $v.MinimalValPath$ as intended. Now we need to complete the algorithm so that it answers what was asked in the question:

```
Algo(G, s)
1: BFS_Modified(G, s)
2: for all v ∈ G.V do
3:     print("NumMinPaths(s, v) = " + v.SPCounter)
4:     print("SumMinPaths = " + v.MinimalValPath)
```

Time-Complexity:

We can easily see that the run-time complexity of $Algo(G, s)$ is $O(V + E)$, this is true because the call at the first line to the function $BFS_Modified(G, s)$ is to a function that is so similar to the function BFS introduced in the lectures, there are some minimal $O(1)$ changes, in a similar way $Init(G, s, Q)$ is also similar to $DFS_Initialization$, thus the total run time in line 1 is $O(V + E)$, the for loop lines 2-4 is a loop iterating all the vertices and printing some values ($O(1)$ operations), hence the run-time of the loop is $O(V)$. In total $T\big(Algo(G, s)\big) = O(V + E)$.

Correctness:

We can easily see that the changes done indeed give us the values we are looking for, this is true because the attribute $SPCounter$ is changed for the first time when the vertex is first discovered (using a shortest path because of the way DFS works), thus the attribute $MinimalValPath$ should be updated then, after that, the attribute $SPCounter$ is updated if and only if the vertex is visited after it was discovered and if the current path length is the same as when it was discovered at the first time.
The attribute $MinimalValPath$ is updated only if the vertex is visited after it was discovered and that the current path length is the same as it was first discovered and if the new sum values in the new path is smaller than the old one.
Thus, after $BFS_Modified$ is terminated, every vertex will have the correct value of attributes. ■